Competency Level 9.12: Manages data in databases

Learning Outcomes:

- Embeds SQL statements in programming languages to retrieve, add, modify and delete data

Python MySQL
Python can be used in database applications. One of the most popular databases is MySQL.

MySQL Database
Download the following free software to experiment with the code examples of MySQL:

MySQL database:	https://www.mysql.com/downloads/
WAMP server:	https://sourceforge.net/projects/wampserver/
XAMPP server:	https://www.apachefriends.org/index.html

Install MySQL Driver
Python needs MySQL driver to access the MySQL database; therefore "MySQL Connector" driver is used. PIP can be used to install "MySQL Connector".PIP is a package manager for Python packages or modules. PIP is most likely installed in the Python environment (When installing a Python, select “Customize installation” instead of “Install Now”. Then PIP will be installed automatically).

Navigate the command line to the location of Python's script directory, and type the following to check whether PIP is installed.

```
C:\Users\your name\AppData\Local\Programs\Python\Python37-32\Scripts> pip --version
```

If PIP is not installed, download and install it from this page:

https://pypi.org/project/pip/

Navigate the command line to the location of Python's script directory, and type the following to download and install "MySQL Connector":

```
C:\Users\your name\AppData\Local\Programs\Python\Python37-32\Scripts>pip install
MySQL-connector
```

Test MySQL Connector
To test if the installation was successful, or if you already have "MySQL Connector" installed, create a Python page with the following content.If the below code was executed with no errors, "MySQL Connector" is installed and ready to be used.

```
import mysql.connector
```

Create Connection
Use the username and password of MySQL database to create a connection

```
import mysql.connector
mydb = mysql.connector.connect(
  host="localhost",
  user="yourusername",
  passwd="yourpassword"
)
print(mydb)
```

Normally default username is “root” and no password

MySQL Create Database
Example 61:
Create a database named "school"

```
import mysql.connector

mydb = mysql.connector.connect(
  host="localhost",
  user="yourusername",
  passwd="yourpassword"
)
mycursor = mydb.cursor()
mycursor.execute("CREATE DATABASE school")
```

MySQL-DeleteDatabase
Example 62:
Delete school database

```
import mysql.connector

mydb = mysql.connector.connect(
  host="localhost",
  user="yourusername",
  passwd="yourpassword"
)
mycursor = mydb.cursor()
mycursor.execute("DROP DATABASE school")
```

MySQL-Create Table
Example 63:
Create student table with: **regNo, name, address and contactNo fields**
Define the name of the database when creating the connection

```
import mysql.connector

mydb = mysql.connector.connect(
  host="localhost",
  user="yourusername",
  passwd="yourpassword",
database="school"
)
mycursor = mydb.cursor()
mycursor.execute("CREATE TABLE student (
regNo VARCHAR(10) NOT NULL,
name VARCHAR(150) NOT NULL,
address VARCHAR(250),
contactNo VARCHAR(10), PRIMARY KEY(regNo))"
)
```

MySQL-Modify Table
Example 64:
Add a new field called "dob" to the student table

```
import mysql.connector

mydb = mysql.connector.connect(
  host="localhost",
  user="yourusername",
  passwd="yourpassword",
database="school"
)
mycursor = mydb.cursor()
```

MySQL- Drop Table
Example 65:
Delete student table

```
import mysql.connector
mydb = mysql.connector.connect(
  host="localhost",
  user="yourusername",
  passwd="yourpassword",
database="school"
)
mycursor = mydb.cursor()
mycursor.execute("DROP TABLE student")
```

MySQL-Insert data into the Table

To fill a table in MySQL, "INSERT INTO" statement is used ***mydb.commit()*** is required to make the changes, otherwise no changes are made the table.

Example 66:

Insert a record in the "student" table

```
import mysql.connector
mydb = mysql.connector.connect(
    host="localhost",
    user="yourusername",
    passwd="yourpassword",
    database="school"
)
mycursor = mydb.cursor()
sql = "INSERT INTO student (regNo, name, address,contactNo, dob) VALUES (%s, %s, %s, %s, %s)"
val = ("r001","Ravi", "Colombo 5", "0715623410","2000-10-21")
mycursor.execute(sql, val)
mydb.commit()
print(mycursor.rowcount, "record inserted.")
```

Output:

```
1 record inserted.
```

('r001', 'Ravi', Colombo 5, '0715874510', '2000-10-21'),

Example 67:

Insert multiple rows

To insert multiple rows into a table, use the **executemany()** method.The second parameter of the executemany() method is a list of tuples, containing the data that want to be inserted.

```
sql = "INSERT INTO student (regNo, name, address,contactNo, dob) VALUES (%s, %s, %s, %s, %s)"
val = [
  ('r002','Mala', 'Anurathapura', '0715874510', '2001-06-14'),
  ('r003','Geetha', 'Kandy', '0775857410', '2001-02-12'),
  ('r004','Kumara', 'Vavuniya', '0710055210', '2000-08-13')
]
mycursor.executemany(sql, val)
mydb.commit()
print(mycursor.rowcount, "was inserted.")
```

Output:

```
3 records were inserted.
```

MySQL Select Data

1. Select all records
2. Select specific columns
3. Select with filter
4. Sort the result

To select from a table in MySQL, use the "SELECT" statement. The **fetchall()** method fetches all rows from the last executed statement.**fetchone()** method fetches only one row.

MySQL Select all records

Example 68:
Select all records from the "student" table, and display the result

```
sql = "INSERT INTO student (regNo, name, address,contactNo, dob) VALUES (%s, %s, %s, %s, %s)"
val = [
  ('r002','Mala', 'Anurathapura', '0715874510', '2001-06-14'),
  ('r003','Geetha', 'Kandy', '0775857410', '2001-02-12'),
  ('r004','Kumara', 'Vavuniya', '0710055210', '2000-08-13')
]
mycursor.executemany(sql, val)
mydb.commit()
print(mycursor.rowcount, "was inserted.")
```

Output:

```
('r001', 'Ravi', ', Colombo 5', '0715874510', '2000-10-21')
('r002','Mala', 'Anurathapura', '0715874510', '2001-06-14')
('r003','Geetha', 'Kandy', '0775857410', '2001-02-12')
('r004','Kumara', 'Vavuniya', '0710055210', '2000-08-13')
```

MySQL Select specific columns

Example 69:
Select only the regNo, name and address columns:

```
mycursor.execute("SELECT regNo, name, address FROM student")
myresult = mycursor.fetchall()
for x in myresult:
  print(x)
```

Output

```
('r001', 'Ravi', 'Colombo 5')
('r002','Mala', 'Anurathapura')
```

Select with filter

When selecting records from a table, selection can be filtered by using the "WHERE" statement

Example 70:
Select record(s) where the birthday is “2001-06-14 “

```
mycursor.execute("SELECT * FROM student WHERE dob='2001-06-14 '")
```

Output

```
('r002','Mala', 'Anurathapura', '0715874510', '2001-06-14')
```

Example 71:
Select records where the contactNo starts with "071"

```
mycmycursor.execute("SELECT name, contactNo FROM student WHERE contactNo LIKE '0710%'")
```

Output

```
('r004','Kumara', 'Vavuniya', '0710055210', '2000-08-13')
```

MySQL Sort the result

Use the ORDER BY statement to sort the result in ascending or descending order

The ORDER BY keyword sorts the result ascending by default. To sort the result in descending order, use the DESC keyword

Example 72:

Sort the result alphabetically by name

```
mycursor.execute("SELECT *  FROM student ORDER BY name ")
```

Output

```
('r003','Geetha', 'Kandy', '0775857410', '2001-02-12')
('r004','Kumara', 'Vavuniya', '0710055210', '2000-08-13')
('r002','Mala', 'Anurathapura', '0715874510', '2001-06-14')
('r001', 'Ravi', ', Colombo 5', '0715874510', '2000-10-21')
```

Example 73:

Display birthdays in descending order

```
mycursor.execute("SELECT name, dob  FROM student ORDER BY dob DESC ")
```

Output

```
('r002','Mala', 'Anurathapura', '0715874510', '2001-06-14')
('r003','Geetha', 'Kandy', '0775857410', '2001-02-12')
('r001', 'Ravi', ', Colombo 5', '0715874510', '2000-10-21')
('r004','Kumara', 'Vavuniya', '0710055210', '2000-08-13')
```

MySQL Update Table

Existing records can be updated in a table by using the "UPDATE" statement

Example 74:

Overwrite the name column from "Ravi" to "Sami".

```
import mysql.connector

mydb = mysql.connector.connect(
  host="localhost",
  user="yourusername",
  passwd="yourpassword",
  database="school"
)
mycursor = mydb.cursor()
sql = "UPDATE student SET name='Sami' WHERE name='Ravi'  "
mycursor.execute(sql)
mydb.commit()
print(mycursor.rowcount, "record(s) affected")
```

Output

```
1 record(s) affected
```

MySQL Delete

Records can be deleted from an existing table by using the "DELETE FROM" statement.

Example 75:

Delete any record where the address is "Kandy"

```
import mysql.connector

mydb = mysql.connector.connect(
  host="localhost",
  user="yourusername",
  passwd="yourpassword",
  database="school"
)
mycursor = mydb.cursor()
sql = "DELETE FROM student WHERE address='Kandy'  "
mycursor.execute(sql)
mydb.commit()
print(mycursor.rowcount, "record(s) deleted")
```

Output

```
1 record(s) deleted
```

Competency Level 9.13: Searches and sorts data

Learning Outcomes:

- Uses sequential searching technique appropriately
- Implements bubble sort technique appropriately

MySQL Searching Techniques

Searching is the algorithmic process of finding a particular item in a collection of items. A search typically answers either *True* or *False* as to whether the item is present.

In Python, there is a very easy way to ask whether an item is in a list of items. We use the *in* operator.

```
>>> 15 in [3,5,2,4,1]
False
>>> 3 in [3,5,2,4,1]
True
```

MySQL Sequential Search

When data items are stored in a collection such as a list, they have a linear or sequential relationship. Each data item is stored in a position relative to the others. In Python lists, these relative positions are the index values of the individual items. Since these index values are ordered, it is possible to visit them in sequence. In this type of search, a sequential search is made over all items one by one. Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data structure.

Sorting Techniques:

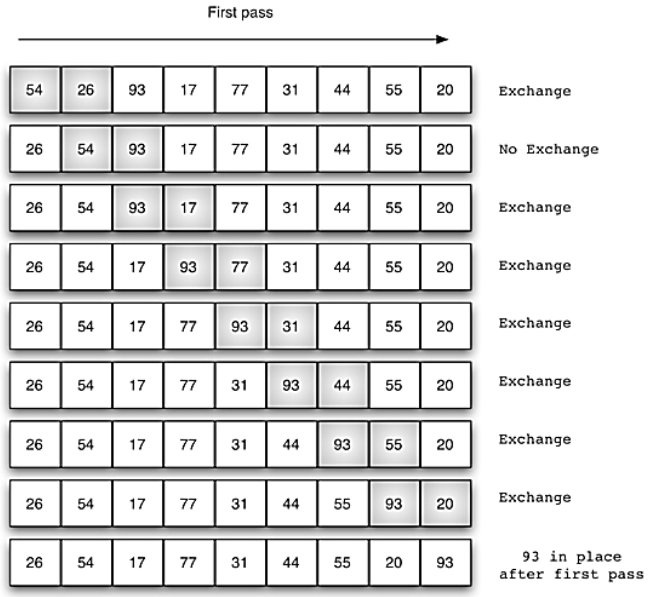
Sorting refers to arranging data in a particular format. It can be ascending or descending order. Sorting algorithm specifies the way to arrange data in a particular order.

Bubble sort:

The bubble sort makes multiple passes through a list. It compares adjacent items and exchanges those that are out of order. Each pass through the list places the next largest value in its proper place. In essence, each item "bubbles" up to the location where it belongs.

Sorting list in ascending order:

- compare 1st and 2nd elements
- if 1st is larger than the 2nd, swap
- compare 2nd and 3rd, and swap if necessary
- continue until compare the last two elements
- largest element is now at the last element in the array
- repeat starting from the beginning until no swaps are needed (i.e.the list is sorted)
- each time you travel the array, elements bubbling up the largest element to the end of the array



At the start of the second pass, the largest value is now in place. There are n−1items left to sort, meaning that there will be n−2 pairs(n is a number of elements in the list). Since each pass places the next largest value in place, the total number of passes necessary will be n−1. After completing the n−1 passes, the smallest item must be in the correct position with no further processing required.

Bubble Sort as a Python Function

```
def bubble_sort(L):
    swapped = True   # set flag to True to repeat sorting
    while swapped:
        swapped = False
        for i in range(len(L) - 1):
            if L[i] > L[i + 1]:
                # Swap the elements
                L[i], L[i + 1] = L[i + 1], L[i]
                # Set the flag to True so we'll loop again
                swapped = True
```

Review Questions

1.
   - Develop a Python program to accept an amount of money, to be paid by a customer in rupees, entered from the keyboard. If the amount is greater than or equal to Rs. 2,000 rupees, a 15% discount is given to the customer. Then display the final amount that the customer has to pay.
   - Modify the above program to display the message "No discount…" if the amount is less than 2,000.
2. Write a program to compute the sum of all even numbers up to 100, inclusive.
3. Develop a Python program to compute the sum of integers between two numbers given as input.
4. Develop a program to display the integers from 100 to 200.
5. Can the loop body of a for loop never get executed?
6. Develop a program to display the following symbol pattern using a nested loop:

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
```